

Explicación práctica y métodos seguidos

En esta práctica se pedía modificar ciertas imágenes del Dataset de MNIST que un modelo sencillo clasificaba mal. Para averiguar qué modificar en la imagen, primero se realizó un pequeño análisis de cómo de bien/mal clasificaba el modelo. Nos dimos cuenta de que el modelo funcionaba bastante mal, careciendo de criterios para clasificar las imágenes. Por eso, tuvimos que recurrir a algunos métodos vistos en clase para analizar las regiones de las imágenes que más destacaban y ver dónde se fijaba el modelo. Este análisis se puede observar en el primer apartado del cuaderno. Solo se han considerado las imágenes que se tenían que cambiar.

Como se observa en el cuaderno, las primeras imágenes sufrían del famoso fenómeno conocido como *vanishing gradients*, ya que vemos que los métodos de explicabilidad basados en gradientes tienen tonos oscuros o no presentan activaciones.

Este fenómeno es clave para la elección de nuestro algoritmo para modificar las imágenes. Si bien podíamos elegir un criterio diferente para las imágenes que sufrían *vanishing gradients* de las que no, decidimos diseñar una manera general de corregir las clasificaciones de todas las imágenes que se pedían.

Para conseguir que el modelo clasifique bien las imágenes que queremos, vamos a realizar un ataque adversarial, pero en vez de tener el objetivo de no clasificar la clase real, para que clasifique bien.

El método iterativo escogido para conseguir esto es el siguiente:

1. Generamos una imagen de ruido gaussiano de media 0 y varianza 1 del mismo tamaño que la imagen de entrada.
2. Creamos dos nuevas imágenes:
 - a. Imagen de entrada + ruido generado en el paso 1
 - b. Imagen de entrada – ruido generado en el paso 1
3. Vemos el *score* que le da el modelo a cada una de estas imágenes y cambiamos la imagen original por la anterior que tenga una puntuación más alta para la clase deseada, añadido con un factor α , preferiblemente mayor que 1 para que el *score* cambie notablemente (en nuestro caso, $\alpha = 2$).
4. Restamos la imagen original con esta imagen y nos quedamos con los top-k píxeles que mayor valor tengan, en valor absoluto (k en este caso representa el número máximo de píxeles que están permitidos cambiar, respetando el límite del 40%).
5. Si esta nueva imagen la predice el modelo como la clase *target*, nos quedamos con esa imagen. Si no, repetimos el proceso hasta que lo clasifique bien.

Con este método, algunas imágenes necesitan solo de unos pocos pasos para que sean clasificadas a la clase *target*, mientras que otras necesitan algunas más, pero finalmente todas acaban clasificando a la clase *target*.

6.